

IssueVault: Hostel Issue Reporting and Monitoring System

A Synopsis for Minor Project

**BACHELOR OF TECHNOLOGY in COMPUTER
SCIENCE & ENGINEERING**

BY
Meena
EN23CS301609

Under the Guidance of
Dr. Sheetal Bawane
Dr. Garima S Tukra



MEDICAPS
U N I V E R S I T Y

**Department of Computer Science &
Engineering Faculty of Engineering
MEDICAPS UNIVERSITY, INDORE- 453331**

Jan - June 2026

1. Introduction

In many hostels and institutional environments, the process of reporting and managing issues is often carried out through informal or manual methods such as verbal communication or written registers.

These traditional approaches are inefficient as they lack proper organization, tracking, and accountability. Users often face difficulties in ensuring that their issues are acknowledged and resolved on time, while administrators struggle to manage multiple complaints effectively.

With the advancement of web technologies, digital solutions have become essential for improving efficiency and transparency.

IssueVault is a web-based issue reporting and monitoring system designed to address these challenges. It provides a structured platform where users can submit issues online, and administrators can monitor and update their status systematically. The system ensures centralized data management, faster communication, and improved tracking of issues, thereby enhancing overall efficiency.

2. Literature Review

The study of issue reporting systems highlights the importance of digital platforms in improving efficiency, transparency, and data management. Various technologies and system designs have been explored to develop effective solutions for handling user-reported issues.

2.1 Digital Issue Reporting Systems

Digital issue reporting systems replace traditional manual methods by providing structured platforms for submitting issues. Users can report problems through predefined forms, which ensures proper data collection and reduces errors. These systems improve efficiency by automating the process and maintaining records systematically.

2.2 Centralized Data Management

Modern systems use centralized databases to store all issue-related information. This allows easy access, retrieval, and management of data whenever required. Centralized storage also ensures consistency and prevents duplication of records.

2.3 Client-Server Architecture

Most web-based applications follow a client-server architecture where the frontend communicates with the backend through APIs. This separation improves system organization, scalability, and maintainability. It also enables multiple users to access the system simultaneously without performance issues.

2.4 Use of Modern Technologies

Technologies such as Node.js and MongoDB are widely used in developing web applications. Node.js provides efficient handling of multiple requests, while MongoDB offers flexible and scalable data storage. These technologies help in building responsive and high-performance systems.

2.5 Status Tracking and Monitoring

Issue tracking systems provide features to monitor the status of reported issues. Users can view updates such as Pending, In Progress, or Resolved. This improves transparency and helps administrators manage issues more effectively.

2.6 Security and Data Validation

Security mechanisms are implemented to protect user data and prevent unauthorized access. Authentication ensures that only valid users can access the system. Input validation helps in maintaining data accuracy and system reliability.

2.7 Limitations of Existing Systems

Many existing systems are designed for large organizations and may be complex for smaller environments. They often include features that are not necessary for basic issue tracking, making them less user-friendly. This creates a need for simpler and more focused solutions.

2.8 Relevance to Proposed System

Based on the above analysis, IssueVault is designed as a simplified and efficient **issue reporting and monitoring system**. It incorporates essential features such as digital reporting, centralized storage, and status tracking. The system is tailored for smaller environments like hostels while ensuring ease of use and scalability.

3. Problem Statement

In many hostel and institutional environments, the process of reporting and handling issues is still carried out using manual or informal methods such as verbal communication or written registers. These approaches are inefficient and lack proper organization, making it difficult to track and manage issues effectively.

As a result, users often face delays in issue resolution and have no clear way to know the status of their reported problems. The absence of a centralized system leads to poor record management, miscommunication, and lack of accountability. Administrators also face challenges in managing multiple issues due to the lack of structured data and tracking mechanisms.

The key problems in the existing system include:

- No proper platform for structured issue reporting
- Lack of centralized data storage for issue records
- Difficulty in tracking the status of reported issues
- Delays in resolving issues due to inefficient processes
- Poor communication between users and administrators

4. Objectives

The main objective of IssueVault is to develop an efficient and user-friendly issue reporting and monitoring system that simplifies the process of reporting, tracking, and managing issues within a hostel or institutional environment.

The system aims to replace manual methods with a structured digital solution that improves transparency, communication, and overall efficiency.

It focuses on ensuring that issues are properly recorded, monitored, and resolved in a timely manner.

The key objectives of the system are:

- To provide a simple and user-friendly platform for users to report issues
- To maintain a centralized and organized database of all reported issues
- To enable administrators to monitor and manage issues effectively
- To allow updating of issue status for proper tracking and transparency
- To reduce manual effort and improve operational efficiency
- To ensure timely resolution of issues through systematic monitoring
- To improve communication between users and administrators

5. Methodology

The development of IssueVault follows a systematic and structured approach to ensure proper implementation of the system.

The methodology is divided into multiple phases, each focusing on a specific part of the development process. This step-by-step approach helps in building the system efficiently while ensuring accuracy, reliability, and ease of maintenance. It helps in identifying issues early and allows continuous improvement throughout the development process.

The methodology includes the following phases:

5.1 Requirement Analysis

This phase focuses on understanding the problem and identifying the requirements of the system.

Both functional and non-functional requirements are defined to ensure clarity in system development.

Key activities include:

- Identifying user needs and system objectives
- Defining features such as issue submission and status tracking
- Analyzing existing problems in manual systems
- Specifying performance and usability requirements

5.2 System Design

In this phase, the overall architecture and structure of the system are planned. The system is designed to ensure smooth interaction between frontend, backend, and database components.

Key activities include:

- Designing client-server architecture
- Planning database schema (issue and user data)
- Defining API endpoints and data flow
- Structuring system modules and components

5.3 Backend Development

This phase involves implementing the server-side logic of the system. The backend processes requests, manages data, and ensures proper communication between components.

Key activities include:

- Developing RESTful APIs using Node.js and Express.js
- Handling request-response operations
- Implementing data validation and basic security mechanisms
- Managing issue-related operations (create, view, update)

5.4 Database Implementation

The database is designed and implemented to store system data efficiently. It ensures proper organization and easy retrieval of issue-related information.

Key activities include:

- Creating collections for issues and users
- Storing fields such as title, description, status, and timestamps
- Ensuring efficient data storage and retrieval
- Maintaining data consistency

5.5 Frontend Development

This phase focuses on developing the user interface of the system. The interface is designed to be simple, interactive, and easy to use.

Key activities include:

- Designing forms for issue submission
- Creating display sections for viewing issues and status
- Applying basic input validation
- Ensuring responsive and user-friendly design

5.6 Integration

In this phase, the frontend and backend are connected to ensure smooth system operation. Data flows between user interface and server through API calls.

Key activities include:

- Connecting frontend with backend APIs
- Sending user input as HTTP requests
- Receiving and displaying responses
- Ensuring proper data flow between components

5.7 Testing and Debugging

The system is tested to ensure that all functionalities work correctly. Errors are identified and fixed to improve system reliability.

Key activities include:

- Performing functional testing of features
- Identifying and fixing bugs
- Validating user inputs and system responses
- Ensuring stable system performance

6. References

1. Node.js Official Documentation, Available at: <https://nodejs.org>,
2. Express.js Official Documentation, Available at: <https://expressjs.com>,
3. MongoDB Documentation, Available at: <https://www.mongodb.com/docs>, accessed on 20 April 2026.
4. Mozilla Developer Network (MDN) Web Docs, Available at: <https://developer.mozilla.org>,

Software Requirement Specification

IssueVault : Hostel Issue Reporting & Monitoring System

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe the requirements of the **IssueVault – Issue Reporting and Monitoring System**.

This document defines the functional and non-functional requirements of the system and serves as a reference for development, testing, and evaluation. It outlines how users will interact with the system and how the system will respond to various inputs. The SRS ensures a clear understanding between stakeholders regarding system behavior and expected outcomes.

It also acts as a foundation for system design and future enhancements.

1.2 Document Conventions

This document follows a structured and standardized format with clearly numbered sections and subsections for easy navigation. All requirements are written in simple and precise language to avoid ambiguity.

The term “shall” is used to indicate mandatory requirements, while “should” is used for recommended functionalities.

Each functional requirement is uniquely identified using a requirement ID for better traceability. Headings, bullet points, and tables are used to improve readability and organization.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Developers (Developers will use this document to understand system requirements and implement functionalities accordingly.)
- Testers (Testers will refer to it for validating system behavior and ensuring all requirements are met.)
- Project evaluators (Evaluators can use it to assess the completeness and clarity of the system design.)

Reading suggestion:

- Section 1 & 2 → Overview of system
- Section 3 & 4 → Detailed requirements
- Section 5 → Quality and constraints

1.4 Product Scope

IssueVault is a web-based **issue reporting and monitoring system** designed to simplify the process of reporting, tracking, and managing issues within a hostel or institutional environment. The system enables users to report issues easily through a structured interface and allows administrators to monitor and update their status efficiently. It aims to improve transparency, reduce manual effort, and ensure timely resolution of reported issues. All issue-related data is stored and managed in a centralized database, making tracking and retrieval more efficient. The system is designed with scalability in mind, allowing future enhancements and broader deployment.

Objectives:

- Provide a platform for students to submit complaints
- Enable admins to manage and update complaints
- Ensure transparency and tracking of issues

Benefits:

- Reduces manual complaint handling
- Improves response time
- Maintains structured records

2. OVERALL DESCRIPTION

2.1 Product Perspective

IssueVault is a standalone web application developed using:

- Frontend: HTML, CSS, JavaScript
- Backend: Node.js, Express.js
- Database: MongoDB

The system is not dependent on any existing application and is designed to function independently. However, it can be integrated with larger systems such as institutional management platforms in the future. The design ensures scalability and flexibility for future enhancements

2.2 Product Functions

The IssueVault system provides several important functions that help users report issues and allow administrators to monitor and manage them effectively.

These functions are designed to ensure smooth communication, proper data handling, and efficient issue tracking within the system.

The major functions of the system include:

- Allowing users to submit issues with relevant details
- Storing issue data securely in the database

- Displaying all submitted issues for monitoring purposes
- Allowing administrators to update the status of issues
- Maintaining records for future reference and tracking

2.3 User Classes and Characteristics

The system is designed to be used by different types of users, each having specific roles and responsibilities. Understanding user characteristics helps in designing a system that is easy to use and efficient for all users. The user classes are:

1. Students:

- Have basic computer knowledge
- Can submit issues and view their status
- Use the system through a simple interface

2. Administrators:

- Have higher privileges and system access
- Responsible for monitoring and updating issues
- Require moderate technical knowledge

2.4 Operating Environment

The system operates in a web-based environment and can be accessed through standard web browsers. It is designed to function efficiently on commonly used operating systems without requiring specialized configurations. The backend runs on Node.js, and MongoDB is used for data storage. A stable internet connection is required for proper communication between client and server.

Operating environment includes:

- **Supported browsers:** Google Chrome, Microsoft Edge, Mozilla Firefox
- **Operating systems:** Windows, Linux (basic compatibility with others)
- **Backend runtime:** Node.js
- **Database:** MongoDB
- **Minimum system requirement:** Standard computer with basic configuration (4GB RAM recommended)
- Internet connectivity required for accessing the system

2.5 Design and Implementation Constraints

The system will be accompanied by basic user documentation to help users understand how to use the application effectively. This documentation will guide users in performing key operations such as submitting issues and viewing their status. It ensures ease of use, especially for users with minimal technical knowledge.

User documentation includes:

- Instructions for submitting issues
- Guidelines for viewing and tracking issue status
- Basic troubleshooting steps
- Description of system features and functionalities
- Simple usage guide for administrators

2.7 Assumptions and Dependencies

The system is developed based on certain assumptions regarding user behavior, environment, and available resources. It also depends on external components and technologies for proper functioning. Any changes in these assumptions or dependencies may affect system performance.

Assumptions include:

- Users have basic knowledge of using web applications
- Users have access to a stable internet connection
- The system will be used within a limited environment (e.g., hostel or institution)

Dependencies include:

- Node.js runtime environment
- MongoDB database availability
- Web browser compatibility
- Proper functioning of internet connectivity
- Availability of system resources during execution

3. EXTERNAL INTERFACE REQUIREMENTS

3.1 User Interfaces

The system provides a simple and user-friendly interface for interaction. The design focuses on simplicity, responsiveness, and ease of use.

Features:

- Complaint submission form with fields:
 - Student Name
 - Issue Title
 - Description
- Button to submit complaint
- Section to view list of complaints
- Display of complaint status
- Error messages and validation prompts for incorrect input
- Responsive layout for different screen sizes

UI Characteristics:

- Clean and minimal design
- Easy navigation
- Proper input validation messages
- Responsive layout for different screens

3.2 Hardware Interfaces

The system does not require any specialized hardware and is designed to work on standard computing devices. It interacts with basic input and output devices available in typical systems. This ensures easy accessibility and usability without additional hardware dependencies.

Hardware interface characteristics include:

- Compatible with standard desktop and laptop systems
- Supports input through keyboard and mouse
- No direct interaction with external hardware devices
- No special hardware configuration required

3.3 Software Interfaces

The system interacts with various software components to perform its operations effectively. These components handle data processing, storage, and communication between different parts of the system. Proper integration ensures smooth functioning and reliable performance.

Software interfaces include:

- **Node.js** for backend runtime environment
- **Express.js** for handling server-side operations and APIs
- **MongoDB** for storing issue and user data
- Web browser for accessing the frontend interface

Data Flow:

- Frontend sends request to backend APIs
- Backend processes request and interacts with database
- Database returns data to backend
- Backend sends response to frontend

3.4 Communications Interfaces

The system uses standard communication protocols to enable interaction between the client and server. All data exchange is performed over the internet using well-defined formats and methods.

This ensures reliable and secure communication.

Communication characteristics include:

- Uses **HTTP/HTTPS** protocol for communication
- Data is transferred in **JSON format**
- Supports API-based communication between frontend and backend

Example:

- POST request → submit complaint
- GET request → fetch complaints
- PUT request → update issue status

4. SYSTEM FEATURES

4.1 User Authentication

4.1.1 Description and Priority

This feature allows users to securely register and log in to the system using valid credentials. The system verifies user identity before granting access to functionalities. It ensures that only authorized users can access specific features, such as updating issue status. User credentials are handled securely to maintain data protection. This feature is important for maintaining system security and controlled access.

Priority: High

4.1.2 Stimulus/Response Sequences

- User enters registration details
- System validates input
- System stores user data in database
- User enters login credentials
- System verifies credentials
- System grants access or shows error message

4.1.3 Functional Requirements

REQ-1: The system shall allow users to register with required details

REQ-2: The system shall validate user input during registration

REQ-3: The system shall store user credentials securely using encryption techniques

REQ-4: The system shall allow users to log in using credentials

REQ-5: The system shall verify user credentials before granting access

REQ-6: The system shall display error messages for invalid login attempts.

4.2 Complaint Submission

4.2.1 Description and Priority

This feature allows users to report issues by providing relevant details such as title and description through a structured form interface. The system ensures that all required fields are properly filled and validated before submission. Once submitted, the issue is stored in the database along with a timestamp for tracking and monitoring purposes. A confirmation message is displayed to inform the user of successful submission.

Priority: High

4.2.2 Stimulus/Response Sequences

- User enters complaint details
- User clicks submit button
- System validates input
- System stores complaint in database
- System displays success message

4.2.3 Functional Requirements

REQ-7: The system shall automatically record the date and time of each complaint submission.

REQ-8: The system shall ensure that all required fields are filled before submission.

REQ-9: The system shall display an error message if submission fails.

REQ-10: The system shall prevent submission of invalid or empty data.

REQ-11: The system shall allow users to enter complaint details

REQ-12: The system shall validate input fields

REQ-13: The system shall store complaint in database

REQ-14: The system shall confirm successful submission

4.3 View Complaints

4.3.1 Description and Priority

This feature allows administrators to view all reported issues in a structured and organized manner. The system retrieves issue data from the database and displays it along with relevant details such as status and timestamp. This helps in monitoring the progress of reported issues and identifying pending tasks. The feature ensures transparency and provides a clear overview of all reported issues.

Priority: High

4.3.2 Stimulus/Response Sequences

- Admin requests complaint list
- System retrieves data from database
- System displays complaints

4.3.3 Functional Requirements

REQ-15: The system shall retrieve complaint data

REQ-16: The system shall display complaint details

REQ-17: The system shall show complaint status

4.4 Update Complaint Status

4.4.1 Description and Priority

This feature enables administrators to update the status of reported issues based on their progress. The system allows status changes such as Pending, In Progress, or Resolved. Only authorized users are permitted to perform this action to maintain system security. The updated status is stored in the database and reflected in the system for users to view. This feature ensures proper monitoring and timely resolution of issues.

Priority: Medium

4.4.2 Stimulus/Response Sequences

- Admin selects a complaint
- Admin updates status
- System saves updated status
- System displays updated result

4.4.3 Functional Requirements

REQ-18: The system shall allow status updates

REQ-19: The system shall store updated status

REQ-20: The system shall reflect updated status in display

5. NON-FUNCTIONAL REQUIREMENTS

5.1 Performance Requirements

The system should provide fast and efficient performance to ensure a smooth user experience. It must handle multiple user requests without significant delay and ensure quick processing of issue data. Performance optimization is important to maintain responsiveness as the number of users increases.

Performance requirements include:

- The system should respond to user requests within 2–3 seconds
- The system should support multiple users accessing it simultaneously
- Database operations should be optimized for fast data retrieval and storage
- The system should maintain consistent performance under normal load conditions
- Delays or failures during issue submission should be minimized

5.2 Security Requirements

Security is an important aspect of the system to protect user data and restrict unauthorized access. The system should ensure that only authorized users can access sensitive functionalities such as updating issue status.

Proper validation and authentication mechanisms should be applied.

Security requirements include:

- The system shall authenticate users before granting access
- The system shall restrict unauthorized access to admin functionalities
- User credentials should be stored securely using encryption techniques
- The system should validate all inputs to prevent malicious data
- Secure communication protocols (HTTP/HTTPS) should be followed

5.3 Quality Attributes

The system should maintain high quality standards to ensure reliability, usability, and maintainability. These attributes help in making the system efficient, scalable, and easy to use for both users and developers.

Quality attributes include:

- Usability: The system should be simple and easy to use
- Reliability: The system should function consistently without failures
- Maintainability: The code should be modular and easy to update
- Scalability: The system should support future expansion (e.g., more users or institutions)
- Performance: The system should provide fast response time
- Flexibility: The system should allow future feature additions
- Testability: The system should be easy to test and debug

6. PROJECT PLAN

6.1 Team Members

This project is being developed individually by a single developer.

S.No.	Name	Enrollment No	Responsibility
01	Meena	EN23CS301609	Requirement analysis, system design, backend development, database design, frontend integration, testing, and documentation

The developer is responsible for complete lifecycle management of the project from planning to final deployment.

6.2 Division of Work

Since the project is developed individually, all tasks will be handled by the developer but organized phase-wise to ensure systematic development. It is divided into the following phases:

- **Requirement Analysis:**
 - Understanding problem statement
 - Identifying functional and non-functional requirements
- **System Design:**
 - Designing system architecture (Frontend, Backend, Database)
 - Creating database schema
 - Planning API structure
- **Implementation (Development):**
 - Backend development using Node.js and Express
 - Database integration using MongoDB
 - Frontend development using HTML, CSS, JavaScript
- **Testing:**
 - Unit testing of individual modules
 - Integration testing of system components
 - Debugging and fixing errors
- **Documentation:**
 - Preparing SRS, SDS, Synopsis, and Report

6.3 Time Schedule

Week	Phase	Description
Week 1	Requirement Analysis	Understanding problem statement and gathering functional & non-functional requirements
Week 2	System Design	Designing system architecture and database schema
Week 3	Backend Setup	Setting up Node.js, Express server, and project structure
Week 4	API Development	Creating basic APIs (POST, GET for complaints)
Week 5	Advanced Backend	Adding update features and improving logic
Week 6	Database Integration	Connecting MongoDB and testing data storage
Week 7	Frontend Development	Building UI using HTML, CSS, JavaScript
Week 8	Frontend–Backend Integration	Connecting frontend with APIs
Week 9	Testing & Debugging	Fixing errors and improving functionality
Week 10	Documentation	Preparing SRS, SDS, Synopsis, and Report

Appendix A: Glossary

- **API:** Application Programming Interface
- **Database:** Structured data storage system
- **Frontend:** User interface of the application
- **Backend:** Server-side logic and processing
- **HTTP:** Communication protocol
- **JWT:** JSON Web Token used for secure authentication
- **User:** Any person who interacts with the system (student or admin)
- **Administrator:** A user with privileges to monitor and update issues
- **Issue:** A problem reported by a user for resolution
- **Status:** Current state of an issue (e.g., Pending, Resolved)
- **Authentication:** Process of verifying user identity
- **Authorization:** Process of granting access to specific functionalities.

- **Validation:** Checking input data for correctness before processing
- **Timestamp:** Date and time when an issue is created or updated

Appendix B: To Be Determined List

- Advanced authentication system (TBD)
- Real-time Notification system (TBD)
- Role-based access control (TBD)
- Image Uploading

Software Design Specification

Software Design Specification for **IssueVault: Hostel Issue Reporting and Monitoring System**
based on event driven (function oriented) technology.

Table of Contents

1. Introduction

- 1.1 Purpose
- 1.2 Document Conventions
- 1.3 Intended Audience and Reading Suggestions
- 1.4 References

2. Analysis Model

- 12.1 Methodology Used (If Structural)
- 12.2 Use Case diagram (Also include use case specifications)
- 12.3 ER Model
- 12.4 Data Flow Diagram (Also write Process Specifications)
- 12.5 Control Flow Diagram (Also write Control Specifications)
- 12.6 State Transition Diagram

3. Design Model

- 13.1 Architectural Design
 - 13.1.1 System Architecture Diagram
 - 13.1.2 Description of Architectural Design
- 13.2 Database Design
 - 13.2.1 Data Dictionary
 - 13.2.2 Normalization
- 13.3 Component Design
 - 13.3.1 Flow Chart
- 13.4 Interface Design
 - 13.4.1 Screenshots

Appendix B1 : Glossary

Appendix B2 : To Be Determined List

1. INTRODUCTION

1.1 Purpose

The purpose of this Software Design Specification (SDS) document is to provide a detailed description of the design structure of the IssueVault – Hostel Issue Reporting and Monitoring System. It defines how the system will be implemented based on the requirements specified in the Software Requirements Specification (SRS).

This document translates functional and non-functional requirements into a structured design that includes system architecture, database design, control flow, data flow, and user interaction design. It serves as a blueprint for developers during implementation and ensures that the system is built in a systematic, organized, and maintainable manner.

1.2 Document Conventions

This document follows a structured and standardized format to ensure clarity and consistency:

- All sections are numbered hierarchically for easy navigation.
- Technical terms are written in standard software engineering terminology.
- Diagrams are referenced clearly under their respective sections.
- “Shall” indicates mandatory design components.
- “Should” indicates recommended design improvements.
- UML and design models are included as visual representations of system structure and behavior.

1.3 Intended Audience and Reading Suggestions

This document is intended for:

- Developers → To understand system architecture and implement modules.
- Testers → To design test cases based on system behavior and flow.
- Project Evaluators → To evaluate system design quality and completeness.
- Future Maintainers → To extend or modify system features.

Reading Flow:

- Section 1 → Overview of SDS
- Section 2 → Analysis Models (system behavior & logic)
- Section 3 → Design Models (architecture & implementation structure)

1.4 References

This SDS is prepared based on:

- Software Engineering textbooks (Pressman – Software Engineering: A Practitioner’s Approach)

- SRS document of IssueVault system
- Node.js, Express.js and MongoDB documentation
- UML design standards
- Web application design principles

2. ANALYSIS MODEL

The Analysis Model represents the functional understanding of the system before design and implementation. It focuses on *what the system should do* rather than *how it will be implemented*.

For the IssueVault system, the analysis model helps in identifying user interactions, system behavior, data flow, and control flow. It acts as a bridge between the Software Requirements Specification (SRS) and the Software Design Specification (SDS).

In simple terms, the analysis model defines:

- System functions
- User interactions
- Data movement
- System behavior over time

It ensures that all requirements identified in the SRS are correctly transformed into a structured system design.

2.1 Methodology Used (Structural Approach)

IssueVault follows a structured analysis and object-based design approach, where the system is decomposed into functional components.

Key points:

- The system is divided into modules like authentication, complaint management, and admin control.
- Each module is analyzed independently.
- UML diagrams are used for modeling system behavior.
- Data flow and control flow are clearly defined.

This approach ensures:

- Better modularity
- Easy debugging and maintenance
- Clear separation of concerns between frontend, backend, and database

2.2 USE CASE DIAGRAM

The Use Case Diagram for IssueVault represents the functional interaction between external users (actors) and the system. It describes how students and administrators interact with the

system to perform different operations such as registration, login, complaint submission, and complaint management.

This diagram focuses on what the system does from a user perspective, rather than internal implementation details. The system is enclosed within a boundary named “IssueVault System”, and all functionalities are represented as use cases inside it.

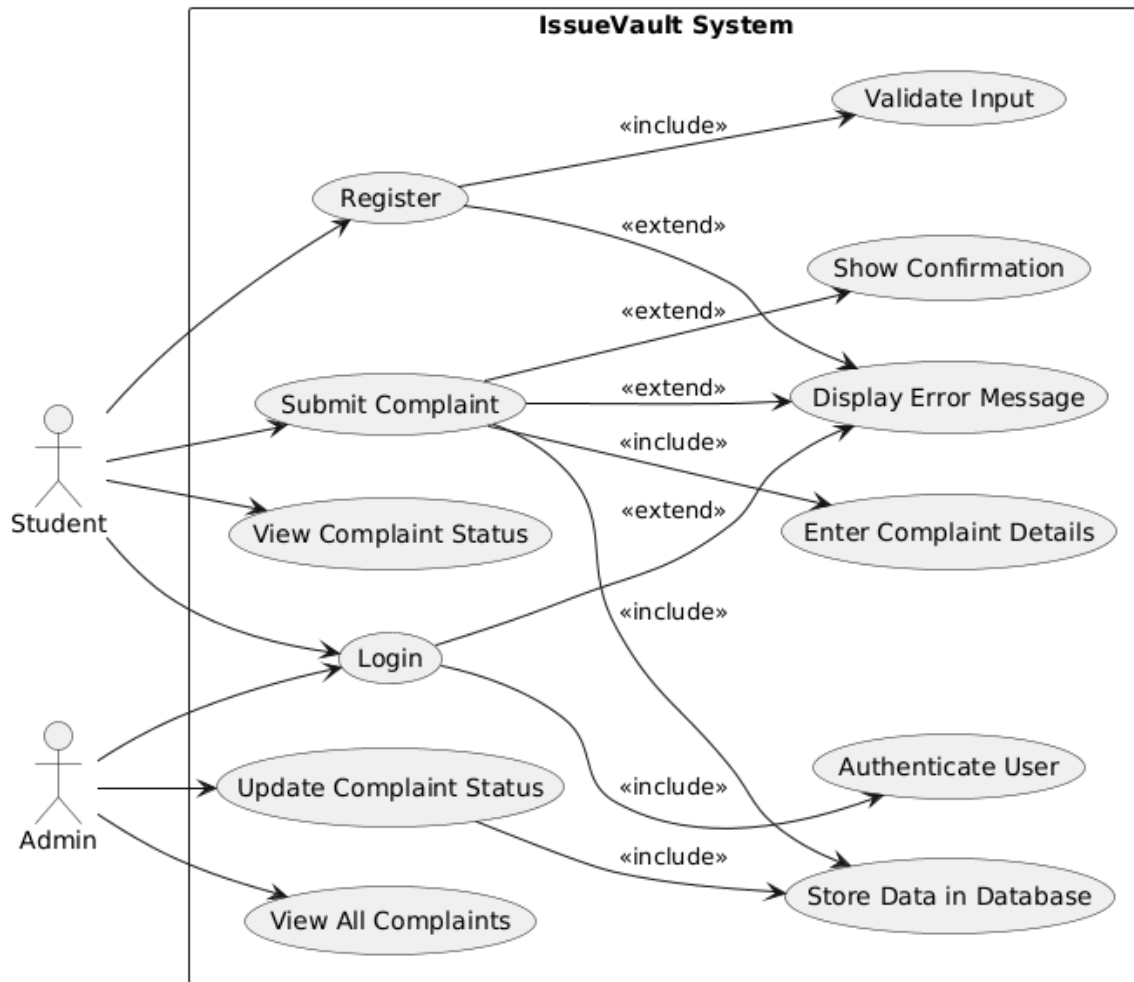


Fig: Use Case Diagram Issuevault

Actors in the System

1. Student

The Student is the primary user of the system who interacts with IssueVault to:

- Register into the system
- Log in to access services
- Submit complaints regarding hostel issues
- View the status of submitted complaints

Students have limited access rights and can only interact with user-level functionalities.

2. Admin

The Admin is the privileged user responsible for managing all complaints. The admin can:

- Log into the system
- View all complaints submitted by students
- Update the status of complaints (Pending, In Progress, Resolved)

The admin has higher-level access and control over system data.

System Use Cases Overview

The system is divided into three major functional categories:

1. Authentication Functions

- Register
- Login
- Validate Input
- Authenticate User

These functions ensure secure access to the system and proper verification of user identity.

2. Student Functions

- Submit Complaint
- Enter Complaint Details
- View Complaint Status

These functions allow students to report issues and track their resolution progress.

3. Admin Functions

- View All Complaints
- Update Complaint Status

These functions enable the admin to monitor and manage all reported issues efficiently.

4. System Functions (Internal Support)

- Store Data in Database
- Display Error Message
- Show Confirmation

These are supporting system-level operations that ensure proper data handling and user feedback.

Relationship:

The Use Case Diagram includes three types of relationships:

1. Association

Shows interaction between actors and use cases:

- Student → Register, Login, Submit Complaint, View Status
- Admin → Login, View Complaints, Update Status

2. Include Relationship

Represents mandatory sub-processes:

- Register → Validate Input
- Login → Authenticate User
- Submit Complaint → Enter Complaint Details, Store Data
- Update Complaint → Store Data

This ensures that core validations and database operations are always executed.

3. Extend Relationship

Represents optional or conditional behavior:

- Register → Display Error Message (if input is invalid)
- Login → Display Error Message (if authentication fails)
- Submit Complaint → Show Confirmation / Display Error Message

This improves system reliability by handling success and failure cases.

2.2.1 USE CASE SPECIFICATIONS

1. Use Case: Register

Field	Description
Actor	Student
Description	Allows new users to create an account in the system
Precondition	User must not already be registered
Main Flow	1. User enters details 2. System validates input 3. Data stored in database 4. Confirmation shown
Alternate Flow	If input is invalid → error message displayed
Postcondition	New user account is created successfully

2. Use Case: Login

Field	Description
Actor	Student / Admin
Description	Allows registered users to access the system
Precondition	User must be registered
Main Flow	1. User enters credentials 2. System authenticates user 3. Access granted
Alternate Flow	Invalid credentials → error message displayed
Postcondition	User is successfully logged in

3. Use Case: Submit Complaint

Field	Description
Actor	Student
Description	Allows students to report hostel-related issues
Precondition	User must be logged in
Main Flow	1. Enter complaint details 2. System validates input 3. Complaint stored in database 4. Confirmation displayed
Alternate Flow	Invalid input → error message displayed
Postcondition	Complaint is successfully recorded

4. Use Case: View Complaint Status

Field	Description
Actor	Student
Description	Allows students to track status of their complaints
Precondition	Complaint must exist in system
Main Flow	1. User requests status 2. System fetches data 3. Status displayed
Postcondition	Current complaint status is shown

5. Use Case: View All Complaints

Field	Description
Actor	Admin
Description	Allows admin to view all submitted complaints
Precondition	Admin must be logged in
Main Flow	1. Admin requests list 2. System retrieves data 3. Displays all complaints
Postcondition	Complaints displayed successfully

5. Use Case: View All Complaints

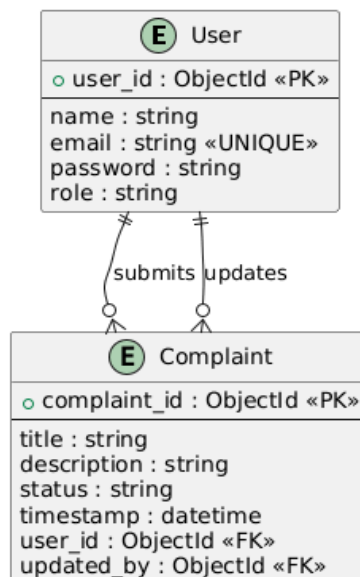
Field	Description
Actor	Admin
Description	Allows admin to update complaint progress
Precondition	Complaint must exist
Main Flow	1. Admin selects complaint 2. Updates status 3. System saves changes 4. Updated status displayed
Postcondition	Complaint status updated successfully

2.3 ER MODEL

The ER diagram of the IssueVault system represents the relationship between two main entities: User and Complaint. It illustrates how users interact with the system by submitting and updating complaints, and how data is structured within the database.

The diagram uses standard ER notation, where:

- Rectangles represent entities
- Attributes are listed inside entities
- Primary keys (PK) and foreign keys (FK) are clearly identified
- Relationships are shown using connecting lines with cardinality



Entity 1: User

The User entity represents all individuals interacting with the system, including both students and administrators.

Attributes:

- user_id (PK) → Unique identifier for each user
- name → Stores the name of the user
- email → Unique email address (ensures no duplicate accounts)
- password → Stores encrypted password
- role → Defines whether the user is a *Student* or *Admin*

Role of Entity:

This entity is responsible for:

- Authentication and authorization
- Identifying the creator of complaints
- Identifying the admin who updates complaints

Entity 2: Complaint

The Complaint entity represents issues reported in the system.

Attributes:

- complaint_id (PK) → Unique identifier for each complaint
- title → Short description of the issue
- description → Detailed explanation of the complaint
- status → Current state (Pending / In Progress / Resolved)
- timestamp → Date and time when complaint is submitted
- user_id (FK) → References the user who submitted the complaint
- updated_by (FK) → References the admin who updated the complaint

Relationships in the Diagram

1. "Submits" Relationship

- A User submits a Complaint
- One user can submit multiple complaints
- Each complaint is submitted by exactly one user

Cardinality: 1 : Many (User → Complaint)

2. "Updates" Relationship

- A User (Admin) updates a Complaint
- One admin can update multiple complaints
- Each complaint is updated by one admin (or none initially)

Cardinality: 1 : Many (User $\rightarrow$ Complaint)

Data Integrity Constraints:

- **Primary Keys ensure uniqueness:**
 - user_id
 - complaint_id
- **Foreign Keys maintain relationships:**
 - user_id $\rightarrow$ links complaint to user
 - updated_by $\rightarrow$ links complaint to admin
- **Unique Constraint:**
 - Email must be unique for each user

2.4 DATA FLOW DIAGRAM

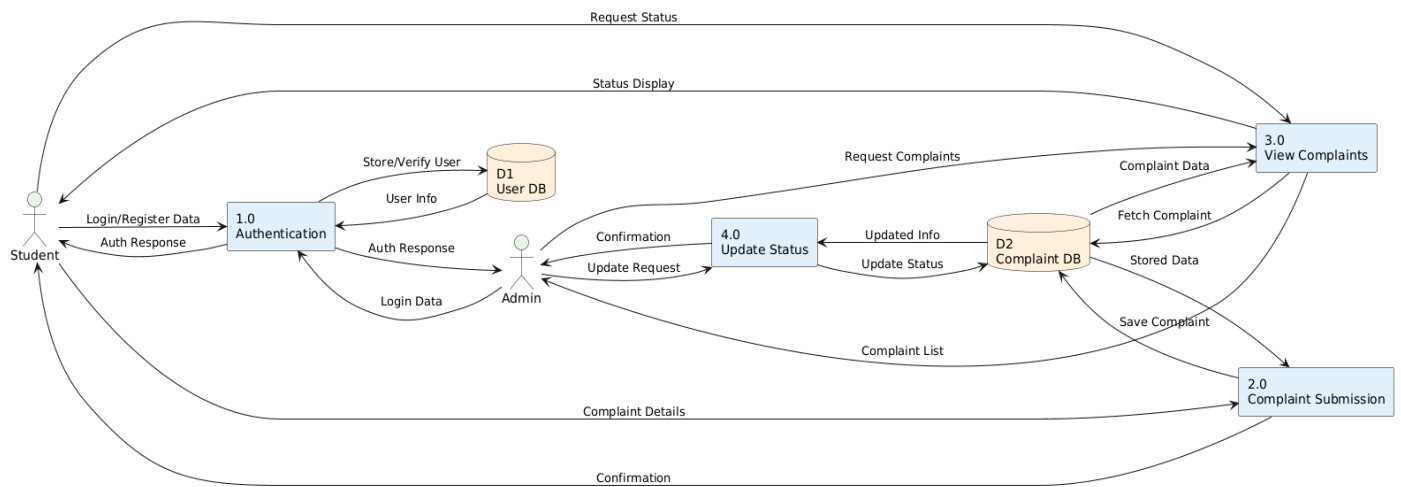


Fig: Data Flow diagram

A Data Flow Diagram (DFD) is a graphical representation of how data moves through a system. It shows:

- How data is input into the system
- How it is processed
- Where it is stored
- How it is output to users

The DFD focuses only on data movement and transformation, not on internal logic or implementation.

In the IssueVault system, the DFD helps in understanding how complaint data flows between:

- Users (Student/Admin)
- System processes
- Databases

DFD Components Used

- **External Entities:** Student, Admin
- **Processes:** Authentication, Complaint Submission, View Complaints, Update Status
- **Data Stores:**
 - D1 → User Database
 - D2 → Complaint Database
- **Data Flows:** Represented using arrows (e.g., Login Data, Complaint Details)

The system is divided into four major processes:

1.0 Authentication Process

This process handles user registration and login.

Data Flow:

- Student/Admin → Login/Register Data → Authentication
- Authentication → User DB (D1) → Store/Verify User
- Authentication → Auth Response → User

Function:

- Validates user credentials
- Stores new user data
- Provides login response

2.0 Complaint Submission Process

This process allows students to submit complaints.

Data Flow:

- Student → Complaint Details → Complaint Submission
- Complaint Submission → Complaint DB (D2) → Save Complaint
- System → Confirmation → Student

Function:

- Accepts complaint input
- Validates data
- Stores complaint in database

3.0 View Complaints Process

This process allows users (Student/Admin) to view complaints and their status.

Data Flow:

- Student/Admin → Request Complaints → View Complaints
- View Complaints → Complaint DB → Fetch Complaint
- Complaint DB → Complaint Data → View Complaints
- View Complaints → Status Display → Student

Function:

- Retrieves complaint data
- Displays complaint status

4.0 Update Status Process

This process allows admin to update complaint status.

Data Flow:

- Admin → Update Request → Update Status
- Update Status → Complaint DB → Update Status
- Complaint DB → Updated Info → Update Status
- Update Status → Confirmation → Admin

Function:

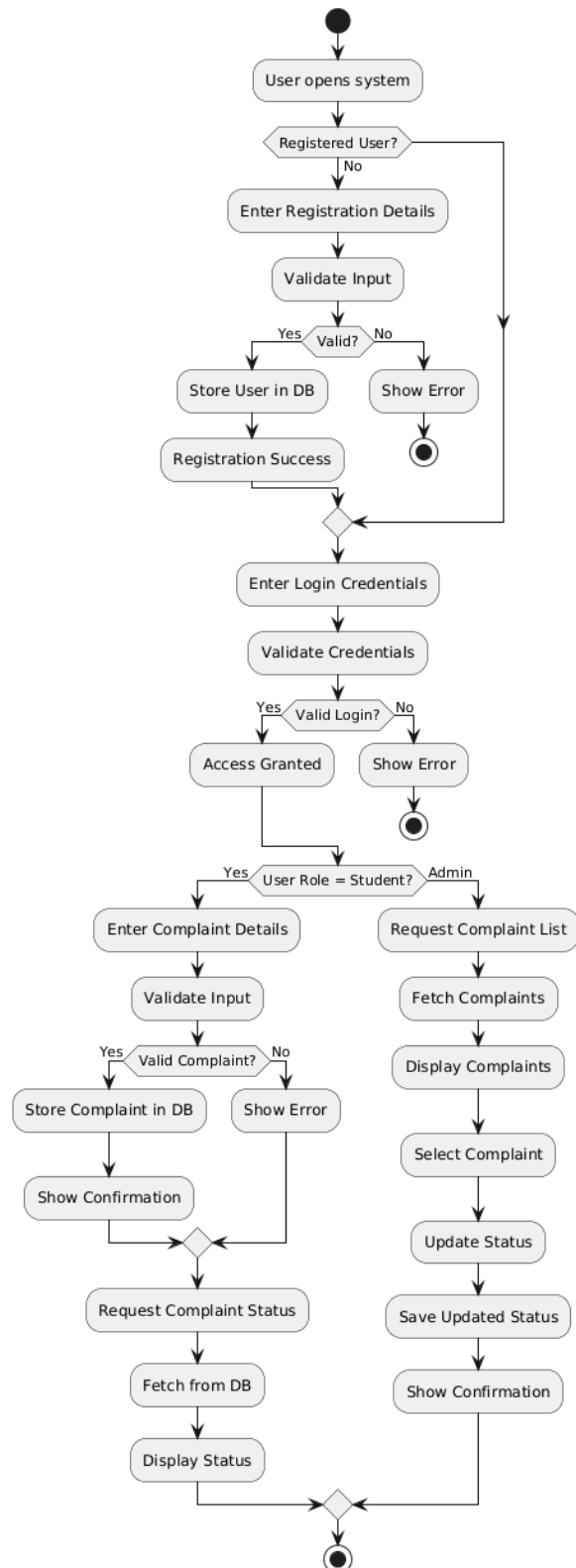
- Updates complaint status
- Stores updated data
- Sends confirmation

2.5 CONTROL FLOW DIAGRAM

The Control Flow Diagram (CFD) of the IssueVault system represents the step-by-step execution flow of the system, starting from user access to final operations such as complaint submission or status update.

It models:

- Sequential execution of processes
- Decision-making using conditional checks
- Role-based branching (Student/Admin)
- Error handling and termination conditions



The diagram ensures that all system operations follow a logical and controlled execution path.

1. System Start

- The control flow begins when the user opens the system.
- This acts as the entry point for all operations.

2. Registration Process (Conditional Entry)

Decision: Registered User?

- If the user is not registered, the system initiates the registration process.

Flow:

1. User enters registration details
2. System validates input

Decision: Input Validity

- If valid:
 - User data is stored in the database
 - Registration success message is displayed
- If invalid:
 - Error message is shown
 - System terminates (stop)

3. Login Process

After registration (or if already registered), the system proceeds to login.

Flow:

1. User enters login credentials
2. System validates credentials

Decision: Valid Login?

- If valid:
 - Access is granted
- If invalid:
 - Error message displayed
 - System terminates

4. Role-Based Control Flow

Once access is granted, the system checks the user role.

Decision: User Role = Student?

5. Student Flow

If the user is a Student, the following operations are performed:

a. Complaint Submission

1. Student enters complaint details
2. System validates input

Decision: Valid Complaint?

- If valid:
 - Complaint is stored in database
 - Confirmation message displayed
- If invalid:
 - Error message displayed

b. View Complaint Status

1. Student requests complaint status
2. System fetches data from database
3. Status is displayed to the user

6. Admin Flow

If the user is an Admin, the following operations are performed:

a. View Complaints

1. Admin requests complaint list
2. System retrieves complaints from database
3. Complaints are displayed

b. Update Complaint Status

1. Admin selects a complaint
2. Admin updates status
3. System saves updated status in database
4. Confirmation message is displayed

7. System Termination

- After completing operations, the system reaches the stop state
- This indicates the end of execution flow

2.6 STATE TRANSITION DIAGRAM

A State Transition Diagram represents the various states of an entity and the transitions between those states based on specific events or conditions. It helps in understanding how the system behaves over time.

In the IssueVault system, this diagram models the lifecycle of a complaint, showing how it moves from creation to completion through different stages.

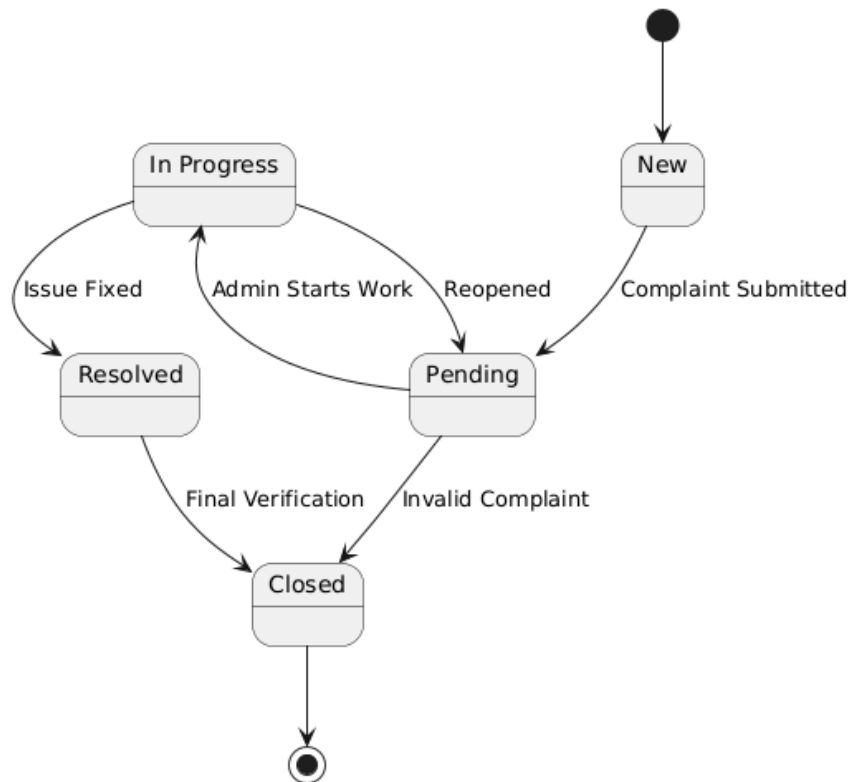


Fig: State Transition Diagram

States Description

- New: Initial state when a complaint is created but not yet processed
- Pending: Complaint is submitted and waiting for admin action
- In Progress: Admin has started working on resolving the issue
- Resolved: Issue has been fixed by the admin
- Closed: Final state where the complaint is verified and completed

State Transitions

- New → Pending: Occurs when the complaint is successfully submitted
- Pending → In Progress: When the admin begins working on the issue
- In Progress → Resolved: When the issue is fixed
- Resolved → Closed: After final verification of the solution

Additional Transitions

- **Pending** → Closed: If the complaint is found to be invalid or not relevant
- **In Progress** → Pending: If the issue is not properly resolved and needs reprocessing

These transitions ensure flexibility and allow the system to handle real-world situations effectively.

3. DESIGN MODEL

3.1 Architectural Design

3.1.1 System Architecture Diagram

Architectural design defines the overall structure of the system and interaction between its components. The IssueVault system follows a 3-tier architecture, which separates the system into Presentation, Application, and Data layers.

This separation improves:

- Maintainability
- Scalability
- Security
- Modularity

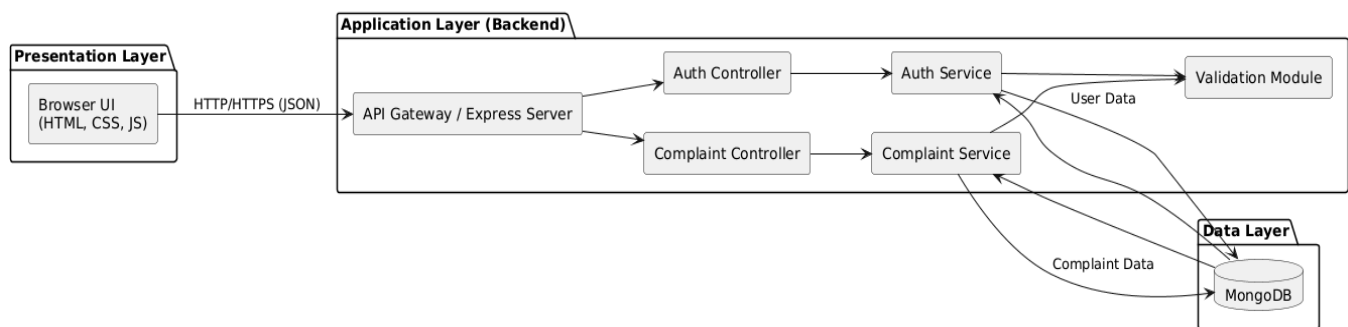


Fig: System Architecture Design

3.1.2 Description of Architectural Design

The IssueVault system is designed using a 3-tier architecture to ensure modularity, scalability, and maintainability. The system is divided into three main layers:

1. Presentation Layer

This layer provides the user interface through which users interact with the system.

- Implemented using HTML, CSS, and JavaScript
- Allows users to perform actions such as login, registration, complaint submission, and status viewing
- Sends requests to the backend using HTTP/HTTPS

- Displays responses received from the server

2. Application Layer (Backend)

This layer handles the core logic and processing of the system.

- Built using Node.js and Express.js
- The API Gateway (Express Server) receives all incoming requests
- **Requests are routed to appropriate controllers:**
 - Auth Controller for authentication-related operations
 - Complaint Controller for complaint handling
- **Controllers interact with:**
 - Auth Service for login and registration logic
 - Complaint Service for complaint processing
- A Validation Module ensures that all inputs are checked before processing

This layered backend structure improves modularity and separation of concerns.

3. Data Layer

This layer manages data storage and retrieval.

- Implemented using MongoDB
- Stores user data and complaint data
- Supports efficient querying and updating of records
- Ensures data integrity through proper structure and relationships

Data Flow in Architecture

The flow of data in the system is as follows:

1. User interacts with the frontend interface
2. Request is sent to the backend server via API
3. Backend processes the request through controllers and services
4. Data is stored or retrieved from the database
5. Response is sent back to the frontend
6. Frontend displays the result to the user

3.2 DATABASE DESIGN

The database design of the IssueVault system defines how data is stored, organized, and managed. The system uses MongoDB, a NoSQL database, to store user and complaint information in a structured manner.

The database is designed to ensure:

- Efficient data storage
- Fast retrieval of records
- Data consistency and integrity
- Scalability for future expansion

Database Schema

The system consists of two main collections:

1. User Collection

This collection stores information about all users of the system, including students and administrators.

Fields:

- user_id → Unique identifier (Primary Key)
- name → Name of the user
- email → Unique email address
- password → Encrypted password
- role → Defines whether user is Student or Admin

2. Complaint Collection

This collection stores all complaints submitted by users.

Fields:

- complaint_id → Unique identifier (Primary Key)
- title → Complaint title
- description → Detailed issue description
- status → Current status (Pending, In Progress, Resolved, Closed)
- timestamp → Date and time of submission
- user_id → Reference to user who created complaint (Foreign Key)
- updated_by → Reference to admin who updated complaint

Relationship Between Collections

- One user can submit multiple complaints
- Each complaint is associated with one user
- Admin actions are tracked using the updated_by field

Relationship Type: One-to-Many (1:M)

3.2.1 DATA DICTIONARY

The Data Dictionary defines the structure and meaning of each data element used in the system.

User Collection

Field Name	Type	Description
user_id	ObjectId	Unique identifier for user
name	String	Name of user
email	String	Unique email address
password	String	Encrypted password
role	String	User type (Student/Admin)

Complaint Collection

Field Name	Type	Description
complaint_id	ObjectId	Unique complaint ID
title	String	Complaint title
description	String	Detailed issue
status	String	Complaint status
timestamp	DateTime	Submission time
user_id	ObjectId	Links to user
updated_by	ObjectId	Admin who updated

3.2.2 NORMALIZATION

Although MongoDB is a NoSQL database, normalization principles are applied to ensure efficient data organization.

First Normal Form (1NF)

- All attributes contain atomic values
- No repeating fields

Applied in both User and Complaint collections

Second Normal Form (2NF)

- All attributes depend on the primary key

Complaint data depends fully on complaint_id

Third Normal Form (3NF)

- No transitive dependencies
- User data is stored separately from complaints

Avoids duplication of user information

3.3 COMPONENT DESIGN

Component Design describes the internal organization of the system by dividing it into smaller functional units called components. Each component performs a specific task and interacts with other components through well-defined interfaces.

The IssueVault system follows a modular component-based design, where frontend, backend, and database components work together to provide complete functionality.

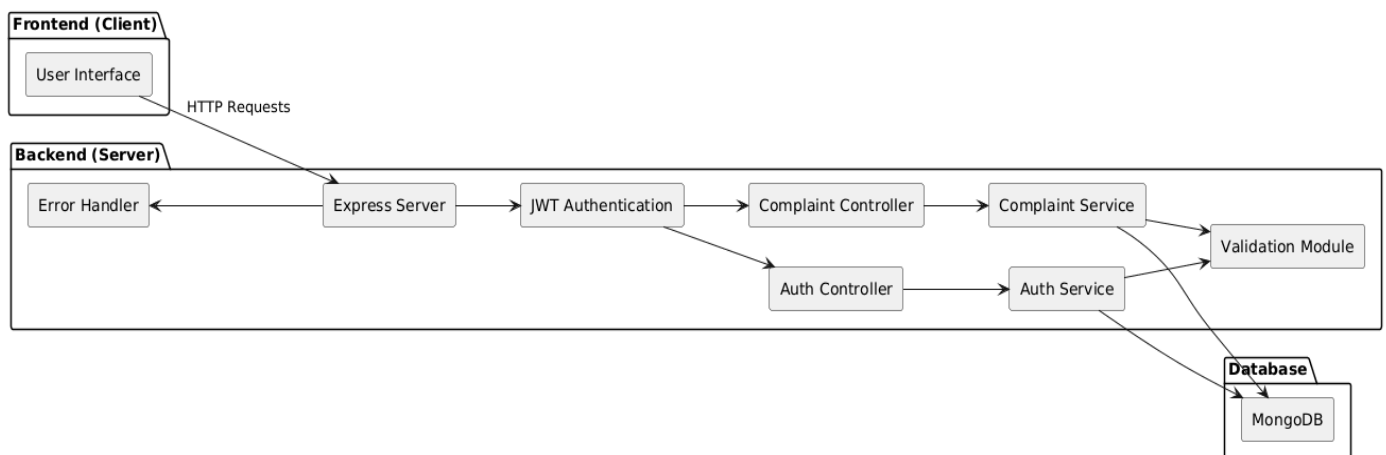


Fig: Component Diagram Issuevault

The diagram illustrates the interaction between frontend, backend modules, and the database.

1. Frontend Component (User Interface)

This component represents the client-side interface through which users interact with the system.

Functions:

- Collect user input (login, complaint submission)
- Send HTTP requests to the server
- Display responses such as status, errors, and confirmations

2. Express Server

Acts as the central server that handles all incoming requests.

Functions:

- Receives HTTP requests from frontend
- Routes requests to appropriate modules
- Integrates authentication and error handling

3. JWT Authentication Component

Handles secure authentication using JSON Web Tokens.

Functions:

- Verifies user identity
- Protects restricted routes
- Allows access only to authorized users

4. Controllers

a. Auth Controller

- Handles login and registration requests
- Passes data to Auth Service

b. Complaint Controller

- Handles complaint-related operations
- Communicates with Complaint Service

5. Services

a. Auth Service

- Processes authentication logic
- Validates user credentials
- Interacts with database for user data

b. Complaint Service

- Handles complaint submission, retrieval, and updates
- Interacts with database for complaint data

6. Validation Module

Ensures that all input data is valid before processing.

Functions:

- Checks required fields
- Prevents invalid or malicious input
- Improves system reliability

7. Error Handler

Handles errors generated during request processing.

Functions:

- Captures runtime errors
- Sends appropriate error responses
- Prevents system crashes

8. Database Component (MongoDB)

Stores all system data.

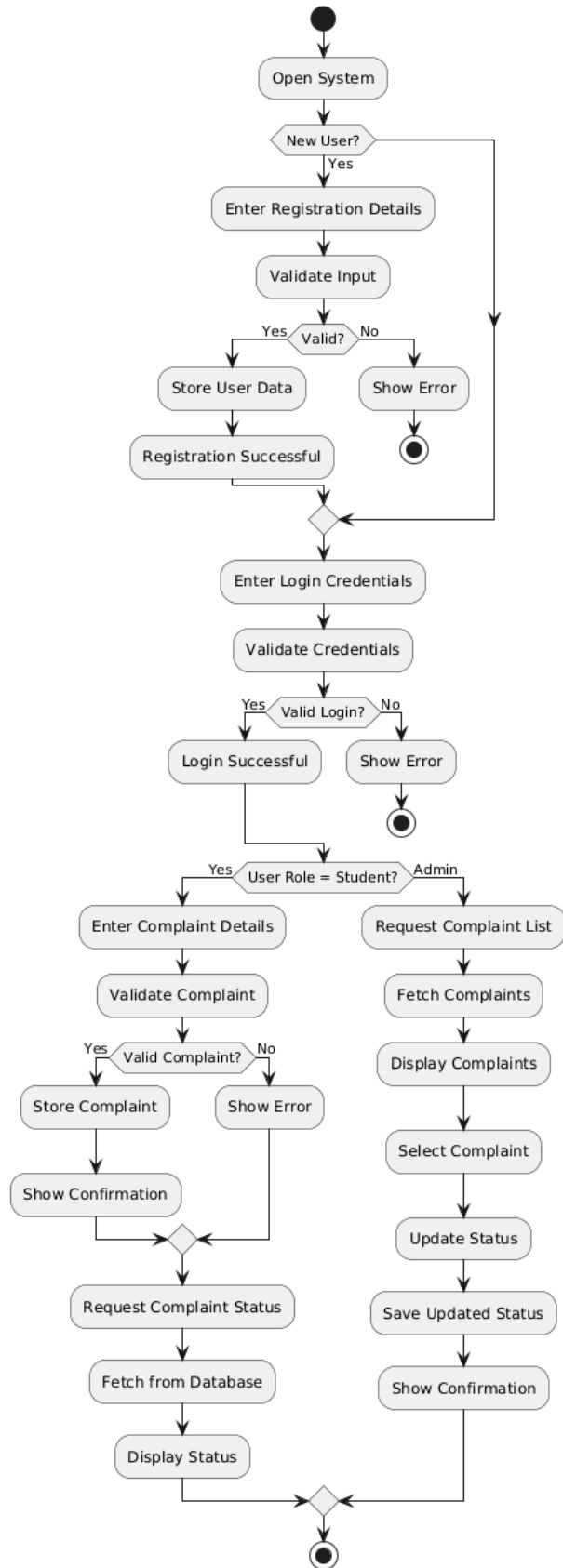
Functions:

- Store user information
- Store complaint data
- Provide data to services when requested

Component Interaction Flow

1. User interacts with User Interface
2. Request is sent to Express Server
3. Request passes through JWT Authentication
4. Server routes request to respective Controller
5. Controller calls appropriate Service
6. Service validates data using Validation Module
7. Service interacts with MongoDB database
8. Response is returned to the user
9. Any errors are handled by Error Handler

3.3.1 FLOWCHART (SYSTEM FLOW)



The flowchart represents the overall working process of the IssueVault system, showing the sequence of operations from system start to completion. It includes user authentication, role-based actions, and complaint management.

Flow Description

1. The process begins when the user opens the system.
2. The system checks whether the user is a new user:
 - If yes, the user enters registration details
 - The system validates the input
 - If valid → user data is stored and registration is successful
 - If invalid → error message is displayed and process stops
3. After registration (or if already registered), the user proceeds to login:
 - User enters login credentials
 - System validates credentials
 - If valid → login successful
 - If invalid → error message and process stops

Role-Based Flow

After successful login, the system checks the user role.

For Student

- User enters complaint details
- System validates complaint
 - If valid → complaint is stored and confirmation is shown
 - If invalid → error message displayed
- User can request complaint status
- System fetches data from database and displays status

For Admin

- Admin requests complaint list
- System fetches and displays all complaints
- Admin selects a complaint
- Updates complaint status
- System saves updated status and shows confirmation

3.4 INTERFACE DESIGN

**IssueVault**
**Raise it.
Resolve it.
Verify it.**
Submit complaints, track their status, and get issues resolved — all in one place.

- ✓ Submit complaints instantly
- ✓ Track status in real time
- ✓ Admin resolves issues fast
- ✓ Secure Hostel ID login
- ✓ Student verifies the issue has solved or not

© 2026 IssueVault

Welcome back 🙌
New here? [Create an account](#)

Hostel ID

Password

Login to IssueVault

Admin? Use your Staff ID to login.

**IssueVault**

Create account ✨
Already registered? [Login here](#)

Full Name

Hostel ID

Password

Role

Create Account

By registering, you agree to use this for hostel purposes only.

My Complaints

Submit a new complaint or track existing ones.

 Pending

1

 Resolved

1

 Verified

1

Submit New Complaint..

Issue Title

e.g. Fan not working in room 204

Description

Describe the issue in detail...

Submit Complaint

Complaint History

All

 Pending

 Resolved

 Verified

Fan not working

The fan in room 204 is not working since 2 days

7 Apr 2026

✓ You verified this complaint

Verified

tubelight issue

tubelight not working in 213 T block from today morning.

8 Apr 2026

✓ You verified this complaint

Verified

tubelight issue

not working for past 2 days

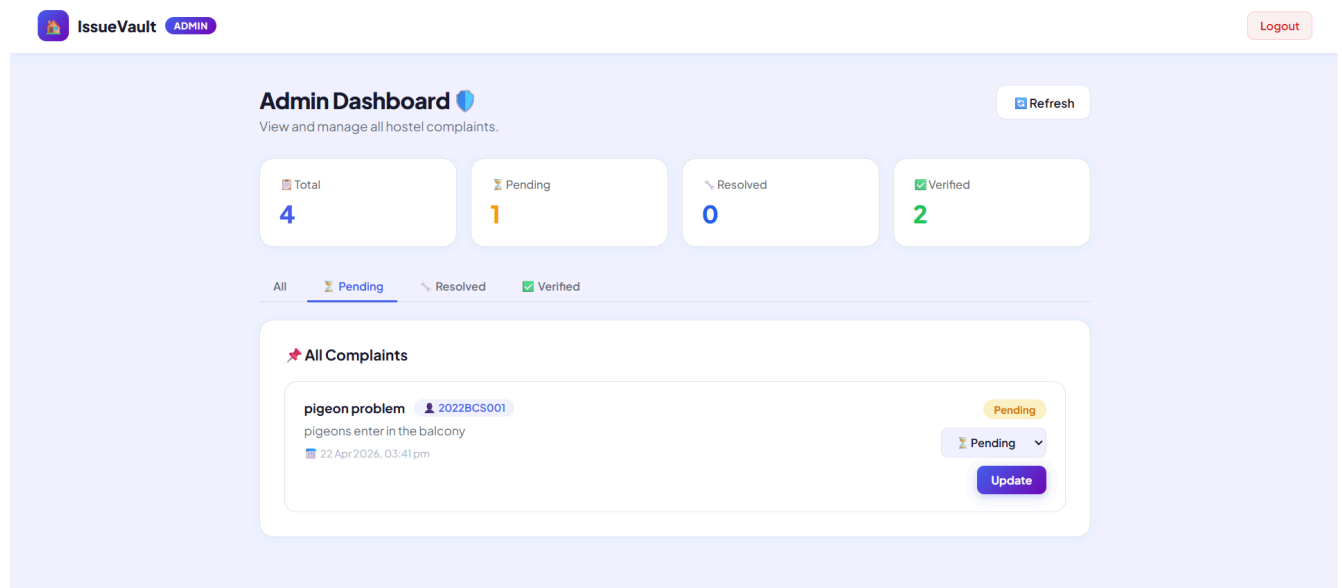
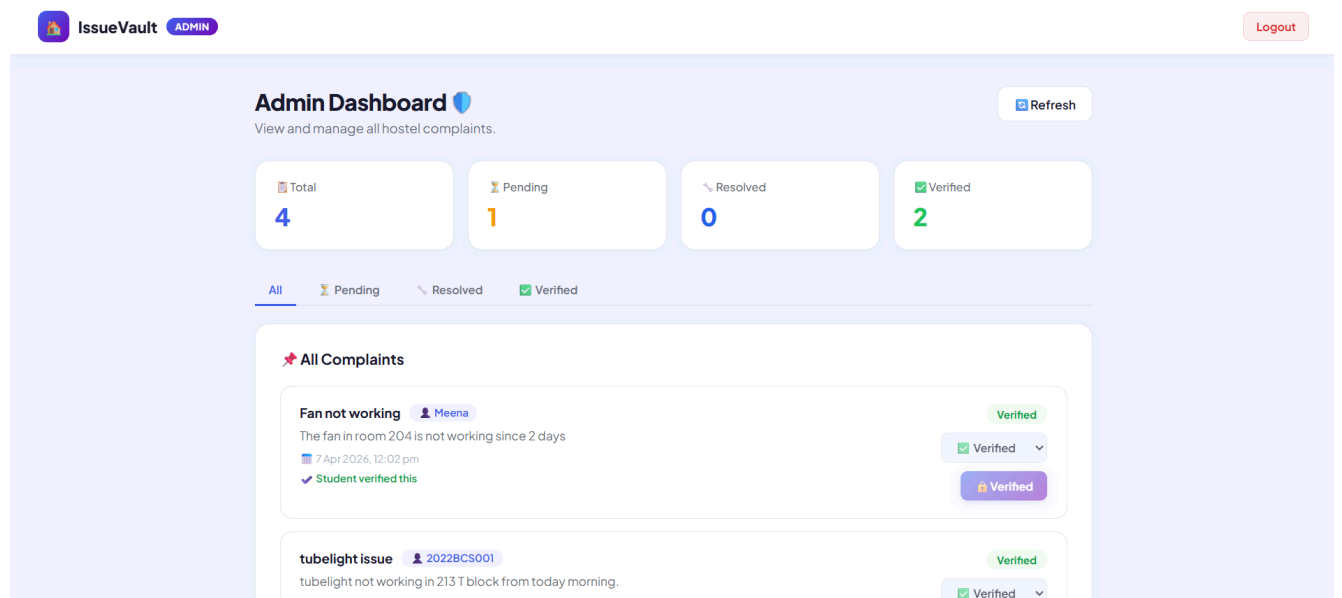
8 Apr 2026

In Progress

pigeon problem

pigeons enter in the balcony

Pending



Appendix A: Glossary

- **API:** Application Programming Interface
- **Database:** Structured data storage system
- **Frontend:** User interface of the application
- **Backend:** Server-side logic and processing
- **HTTP:** Communication protocol
- **JWT:** JSON Web Token used for secure authentication

- **User:** Any person who interacts with the system (student or admin)
- **Administrator:** A user with privileges to monitor and update issues
- **Issue:** A problem reported by a user for resolution
- **Status:** Current state of an issue (e.g., Pending, Resolved)
- **Authentication:** Process of verifying user identity
- **Authorization:** Process of granting access to specific functionalities.
- **Validation:** Checking input data for correctness before processing
- **Timestamp:** Date and time when an issue is created or updated

Appendix B: To Be Determined List

- Advanced authentication system (TBD)
- Real-time Notification system (TBD)
- Role-based access control (TBD)
- Image Uploading